Proving What the Model Inferred: A Survey of zkML

Philippe Laporte
ID# 24172957

Recent Developments in Information Security

SNARKs and Zero Knowledge Technology

Information Systems Engineering

Concordia University

April 23 2026

Preamble

This report owes its inspiration to the convergence of 3 topics with which I have acquainted myself this past academic year: Applied Machine Learning, Blockchain Technologies, and Advanced Cryptography/Zero Knowledge Proofs. It follows a theme introduced in the previously submitted Use-Case Study

Succinct's Platform, Prover Network and SP1

as well as my ramping up knowledge on the Rust Programming Language, the most popular choice of ZKP programming frameworks, including the EZKL project to which I contributed in the context of this report. Rust is well-known for Safety, speed and fearless concurrency.

Introduction

Zero-knowledge machine learning has crossed from research curiosity to functioning engineering discipline in four years. In 2022 Kang et al. first proved a full-resolution ImageNet classifier with ten-second verification; in 2024 zkLLM reached a 13B-parameter LLaMA-2 forward pass in fifteen minutes; in 2025 ZKTorch delivered the first universal zkML compiler; and EZKL now powers production on-chain ML verification for credit scoring, consensus mechanisms, and verifiable AI contests on Ethereum. The question is no longer whether zkML is possible, but how to make it post-quantum, hardware-accelerated, and economically competitive with probabilistic alternatives. This report surveys the academic lineage from SafetyNets through ZKTorch, the 2025–2026 state of the art, the quantum-safety picture across the proof-system stack, the CPU-versus-GPU hardware trajectory, the circuit-complexity gulf between neural and classical ML, probabilistic verification schemes like Freivalds' algorithm and cut-and-choose, EZKL's role as the bridge between zkML and Ethereum's on-chain verification infrastructure, and a concrete open-source contribution — PR #1025 to EZKL, addressing the float-to-field quantization boundary that defines the field's central engineering challenge.

Machine Learning: A Primer for the ZKP Practitioner

Before surveying the proof systems, it helps to be precise about what "machine learning" is and which of its internal distinctions drive the enormous variation in proving cost that the rest of this report documents. Machine learning is, at its core, the problem of finding a function $M : X \to Y$ that maps inputs to outputs by learning patterns from data rather than being explicitly programmed. The field splits into two broad paradigms — statistical (or classical) ML and neural-network-based (deep) ML — and two distinct computational phases — training and inference. Each of these four quadrants has radically different implications for zero-knowledge proofs.

Statistical (classical) machine learning

Statistical ML refers to a family of algorithms grounded in classical statistics and optimization theory. The models are typically compact, interpretable, and defined by a small number of learned parameters relative to the data they were trained on. The key model families are linear and logistic regression (a single dot product plus at most one nonlinearity), decision trees (a sequence of axis-aligned feature comparisons with no multiplication), random forests and gradient-boosted trees such as XGBoost and LightGBM (ensembles of hundreds of independent trees whose predictions are averaged or majority-voted), support vector machines (a dot product for linear SVMs, or per-support-vector kernel evaluations for nonlinear SVMs), and k-nearest neighbors (distance computation and sorting over the training set, with no learned parameters).

What unites these models from a ZKP perspective is that their inference operations are dominated by field-native arithmetic: additions, multiplications, and comparisons over integers or fixed-point numbers. The nonlinearities, when present, are mild (a single sigmoid, a comparison at each tree node). The parameter counts are small — a logistic regression on 1024 features has 1025 parameters; a random forest of 500 trees with 31 nodes each has roughly 15,500 decision nodes total. These properties make statistical ML dramatically cheaper to prove in a ZK circuit than neural networks.

Neural networks (deep learning)

Neural networks are parameterized function approximators built by composing many layers of linear transformations and nonlinear activation functions. The key families are multilayer perceptrons (MLPs — alternating matrix multiplications and pointwise activations like ReLU), convolutional neural networks (CNNs — learned spatial filters, 25M–138M parameters for ResNet-50/VGG-16), and transformers/LLMs (GPT, LLaMA — built around self-attention, where $\text{softmax}(QK^T/\sqrt{d_k})\cdot V$ is the single most ZK-hostile operation in modern ML, requiring exponentiation, summation, and division with no prime-field representation; GPT-2 has 1.5B parameters, LLaMA-2-13B has 13B).

Inference involves enormous matrix multiplications (>90% of FLOPs), repeated nonlinear activations (none polynomial over a prime field), floating-point arithmetic (requiring fixed-point quantization for field compatibility), and very large parameter counts. These properties explain why proving a neural-network forward pass costs 10^4–10^6× more constraints than an equivalent statistical-ML prediction.

Training vs. inference: two fundamentally different computations

Training iteratively adjusts parameters θ to minimize a loss function via stochastic gradient descent — millions of steps, each a forward pass plus a backward pass (chain-rule gradient computation). Proving training means proving a very long sequential computation: total cost scales as $O(S \times C_{step})$, and even at 1 second per step, 1 million steps take 11.5 days. Inference is a single forward pass of a committed model on a new input — one function evaluation with no gradients or parameter updates.

This asymmetry is why proof-of-inference systems (zkCNN, zkLLM, EZKL, ZKTorch) are far more mature than proof-of-training systems (zkPoT, Kaizen, zkDL), and why probabilistic shortcuts like cut-and-choose (Section 7) are appealing for training verification.

A map of the proving-cost landscape

Combining the two axes — model type and computation phase — produces a rough landscape of ZKP cost that motivates the entire rest of this report:

	Statistical ML	**Neural network (small)**	**Neural network (large)**
Inference	Cheap: dot product + comparison. 250s for a 1029-node tree (zkDT).	Moderate: 88s for VGG-16 (zkCNN).	Expensive: 15 min for 13B LLaMA (zkLLM).
Training	Feasible: 10 min for 262k-sample logistic regression (zkPoT).	Hard: <1s per step for 10M params (zkDL), but millions of steps.	Currently impossible at full scale.

The rest of this report follows the field's efforts to push the "expensive" and "hard" cells toward practicality — through better proof systems, hardware acceleration, probabilistic shortcuts, and architecture-aware circuit design.

A Short History: From Hand-Crafted Circuits to Universal Compilers

SafetyNets [1] recognized that a neural network is a layered arithmetic circuit — the structure the GKR protocol [2] verifies. GKR reduces "the output is y" layer-by-layer via sumcheck: at each layer, a sumcheck over a multilinear polynomial encoding the wiring reduces the claim to the layer below. After L reductions the claim reaches the known inputs. SafetyNets was limited to quadratic activations (x^2) since GKR handles only arithmetic gates, but its layer-by-layer sumcheck template became the backbone of zkCNN, Libra, zkLLM, zkGPT, DeepProve, and EZKL.

ZEN [3] achieved 5–22× constraint reductions via three float-to-field techniques: sign-bit grouping (amortizing ~254-constraint sign checks across outputs sharing sign patterns), remainder verification (exploiting power-of-two divisors for compact fixed-point rescaling proofs), and SIMD stranded encoding (packing $\lfloor 254/b \rfloor$ quantized b-bit values per field element). These established the quantization playbook that EZKL, ZKML, and zkLLM all descend from.

zkCNN [4] proved VGG-16 (138M-parameter CNN) in 88 seconds with a 341 KB proof using a sumcheck for FFT-based convolutions with linear prover time. Mystique [5] reached ResNet-101 in 28 minutes via VOLE-based interactive ZK — linear-cost proving without FFTs or trusted setup, but with interactive, large proofs unsuited to on-chain verification.

The second phase produced general-purpose compilers. Kang et al. [6] demonstrated the Halo2-plus-lookup recipe underpinning EZKL, still the dominant ONNX-to-proof pipeline. ONNX (Open Neural Network Exchange) is the framework-agnostic format that PyTorch, TensorFlow,

and scikit-learn all export to, making it the de facto zkML front-end. ZKML [7] was the first optimizing Halo2 compiler for GPT-2 with ≤ 0.01% accuracy loss.

The third phase reached LLM scale. zkLLM [8] proved 13B parameters via tlookup (batched tensor-level lookups reducing per-element cost from O(log n) to near-constant), zkAttn (digit-decomposition softmax avoiding exp() approximation), and GPU-parallelized GKR sumcheck. ZKTorch [9], the first universal zkML compiler, proves GPT-J 6B in ~20 minutes and shrinks ResNet-50 proofs from 1.27 GB to 85 KB.

The 2026 Landscape: Frameworks, Scaling Tools, and Infrastructure

Five frameworks define the frontier. zkLLM [8] holds the largest single-forward-pass benchmark at LLaMA-2-13B (~15 minutes on an A100, ~80 GB peak memory). ZKTorch [9] generalizes across architectures but caps around 6B. zkPyTorch [10] offers a PyTorch-native frontend on Polyhedra's Expander GKR engine. zkGPT [11] reports GPT-2 proofs in under 25 seconds. zkLoRA (2025) is the first end-to-end ZKP of LoRA fine-tuning.

The gap between 13B parameters proven and 100B+ deployed is bottlenecked by memory (~80 GB polynomial commitments), nonlinear-op cost (softmax/GELU/layer-norm scale poorly), and circuit size (tens of trillions of constraints for 100B+ naive compilation). Four tool categories are being stacked to close it. Folding schemes (Nova [17], HyperNova [18], ProtoStar [19]) incrementally compress proof instances into a running accumulator — for zkML, 2,000-token autoregressive generation requires one final proof rather than 2,000 separate ones. Distributed provers (DIZK [20], Pianist [21]) split proving across machines, with Pianist achieving linear speedup and constant proof size via distributed polynomial commitments. Specialized sumcheck and GKR (Libra [22], Lasso [23]) reduce per-layer cost — Lasso's virtual tables commit $m + \sqrt{T}$ elements for million-entry activation tables. Hardware acceleration is covered in Section 5.

The zkVM alternative proves general programs by compiling inference to a VM ISA. Over 20 implementations span RISC-V (SP1, RISC Zero, Jolt [23], Brevis Pico, OpenVM, Valida), Cairo (Giza's Orion, LuminAIR on Stwo), WASM, MIPS (ORA's opML reaching 7B-LLaMA via fraud-proof bisection), and custom ISAs. All remain 10–100× slower than specialized compilers for ML; whether zkVM overhead drops below 5× will determine the field's standardization path.

Three proof modalities are defined: inference [6,4,8,9] ($y = M(x)$ for committed M), training [12,13] (M produced by algorithm A on dataset D; zkDL [14] proves single steps in under a second), and prompt [15] (private prompt bound to public output). A related strand — verifiable AI watermarking [16] — enables proving correct model execution and correct watermark insertion inside the same circuit.

Decentralized prover networks

Decentralized prover networks route proof jobs to GPU supply via economic incentives. The Succinct Prover Network (mainnet August 2025) uses a stake-and-slash model: provers stake

PROVE tokens as collateral, earn payment on valid delivery, and are slashed on timeout or invalid proof. Other networks include Gevulot, the =nil; Proof Market, Fermah, Kalypso, Aligned Layer (an EigenLayer AVS aggregating proofs from SP1, RISC Zero, PlonK and Halo2), and Cysic. ML workloads create unusual demands — 80+ GB VRAM, minutes-long jobs — requiring higher collateral thresholds than lightweight proofs.

Production deployments and on-chain AI

Worldcoin's Remainder — a GKR+Hyrax prover — is the field's only at-scale continuous deployment, generating iris-embedding inference proofs on-device. Giza's Orion runs on Starknet. ORA's opML deploys optimistic-ML oracles on Ethereum mainnet. Spectral Finance uses EZKL for on-chain credit scoring. Ritual's Symphony consensus exposes four verification sidecars (ZKML, opML, PPML probabilistic checking, and TEE attestation), letting developers dial verification guarantees against cost.

Quantum Safety: The Wrapper Is the Weak Link

STARKs (FRI-based [24], hash-committed) rely only on collision-resistant hashes and are plausibly post-quantum secure — Grover gives only a √ speedup, absorbed by 256-bit outputs. Pairing-based SNARKs (Groth16 [36], PLONK+KZG, Marlin) and Bulletproofs reduce to discrete log and are broken by Shor's algorithm. Every major zkML system deployed on Ethereum today is quantum-vulnerable at its on-chain layer:

Framework	Core proof	Wrapper	Quantum-safe?
EZKL (Zkonduit)	Halo2 + KZG on BN254	—	No
zkLLM	Tensor sumcheck + Hyrax on BLS12-381	—	No
RISC Zero	FRI over BabyBear	Groth16 on BN254	Core yes, wrapper no
SP1	Plonky3 (FRI)	Groth16/PLONK	Core yes, wrapper no
ZKTorch	Mira accumulation (pairing)	—	No
Giza Orion (Stwo)	FRI STARK	Native	Yes (until wrapped)

The STARK-then-SNARK wrapping used by RISC Zero and SP1 is the crux: both produce a PQ-plausible STARK internally, then verify it inside a Groth16 circuit so Ethereum's BN254 precompile can check the final proof for ~250K gas rather than the 2–5M gas a native FRI verifier would cost. The on-chain artifact inherits only the wrapper's strength — a quantum adversary could forge a Groth16 proof for any false zkML claim.

The PQ proof-system research menu includes hash-based systems (FRI [24], DEEP-FRI [25], STIR [26], WHIR [27], Binius [28], Ligero [29], Aurora [30], Brakedown [31], Orion [32]) and lattice-based approaches (LaBRADOR [33], Greyhound [34], LatticeFold [35]), though the latter carry 10–100× overhead and immature GPU tooling. For long-horizon zkML applications — medical records, legal attestations, Worldcoin's iris claims — the prudent architecture keeps the

authoritative proof as a STARK and treats any Groth16 wrapper as a liveness convenience rather than durable cryptographic truth.

Hardware: CPU vs. GPU Proving, and the Road to ZK Silicon

Why proving is GPU-bound and verification is CPU-bound

Proving is dominated by multi-scalar multiplication (MSM, ~60% of runtime in Groth16) and number-theoretic transforms (NTT, ~30%), both massively data-parallel. All ZK arithmetic is integer-only — modular multiplication over prime fields — so GPU floating-point units, tensor cores, and ray-tracing silicon sit idle. The real bottleneck is memory bandwidth: MSM requires enormous random accesses into point tables, and NTT requires large data movement between butterfly stages, which is why the bandwidth gap between CPU (DDR5, ~100 GB/s) and GPU (GDDR7 on RTX 5090, ~1.79 TB/s) translates directly into proving speedups.

Verification is the opposite: O(1) for Groth16 (three pairing checks, 1.2–1.8 ms on one CPU core), O(log N) for PlonK/Halo2, O(polylog N) for FRI-STARKs. GPU kernel launch overhead alone is a significant fraction of total verification time. On-chain verification locks in the CPU model — Groth16 costs ~200–250K gas, raw FRI costs ~5–18M gas (prohibitive), which is why every production STARK stack wraps into a pairing-based SNARK for EVM verification via Ethereum's BN254 precompiles.

This asymmetry is ideal for zkML: the prover needs throughput (datacenter GPUs, seconds to minutes), the verifier needs latency and universality (any smartphone, any smart contract, milliseconds, cents).

Measured GPU speedups: large per-kernel, modest end-to-end

Per-kernel speedups are dramatic (63× MSM, 320× NTT on RTX 4080 via ICICLE), but Amdahl's law on the CPU-resident tail — witness generation, Fiat-Shamir hashing, constraint evaluation, PCIe transfer — collapses these to single-digit end-to-end multipliers: cuZK [43] reports 2.65× overall Groth16, GZKP [44] sees 4–20× on real workloads despite 48× on isolated benchmarks. The sumcheck protocol compounds the problem — each round depends on the previous round's verifier challenge, serializing across rounds and capping GPU parallelism even within GKR-based systems like zkLLM.

The determinism advantage: why ZK never uses floating-point

ZK circuits are evaluated over finite fields — modular integers — which are bit-exact deterministic by construction. This sidesteps the entire GPU floating-point non-determinism problem (reduction-order variance, fused-multiply-add rounding) that plagues other approaches to verifiable ML. The cost is that every floating-point operation in the ML model must be quantized to fixed-point integer field arithmetic before proving. EZKL applies user-tunable fixed-point scaling factors; ZKML [7] starts from pre-quantized int8 TFLite models; zkLLM

uses quantization level Q=16 (values in [0, 65536]) with lookup-based bounds checking; ZKTorch inherits the same integer-only pipeline. This quantization step is itself CPU-bound and contributes to the Amdahl's-law ceiling on GPU speedups.

Consumer GPUs now beat datacenter hardware for ZK

Because ZK workloads are integer-bound and memory-bandwidth-bound (not tensor-core-bound), consumer GPUs beat datacenter GPUs on $/performance — the RTX 5090's GDDR7 bandwidth at one-tenth the H100's price dominates, and both Brevis Pico Prism and Succinct SP1 Hypercube independently proved 99%+ of Ethereum blocks in under 12 seconds on 16 RTX 5090s (~$100K hardware), down from 64–200 GPUs six months earlier. Apple Metal support via ICICLE v3.6 enables on-device zkML proving on M-series chips — critical for biometric and medical inference where data cannot leave the device.

FPGA, ASIC, and the specialization trajectory

ZK-specific silicon is arriving: Cysic's C1 ASIC (~13× over RTX-class GPUs), Fabric Cryptography's VPU (40 custom tiles, >1 TB/s on-chip NoC), Ingonyama's ZPU, and Irreducible's Binius-optimized design all target 10–50× over GPU, with ASICs requiring sustained network-wide proving volume projected to arrive by 2027. The convergence toward real-time zkML stacks four effects: sumcheck-native protocols mapping to data-parallel hardware, small-field STARKs cutting arithmetic costs 4–8×, purpose-built silicon, and multi-GPU clustering as demonstrated by Pico Prism's 16-GPU architecture.

Neural Networks vs. Classical ML: The Circuit-Friendliness Gulf

ZK proof cost is dominated not by FLOPs but by constraint count and the prevalence of non-arithmetic operations that must be emulated via range checks, bit decomposition, or lookup tables. That single fact creates an enormous gradient in ZKP-friendliness between neural networks and classical statistical ML — and it is the central engineering challenge that frameworks like EZKL must solve at every layer of their stack.

Weight quantization: the float-to-field bridge

Every zkML system must convert floating-point model weights and activations into prime-field integers. A ZK circuit operates over $\mathbb{F}_p$ — modular integers with only addition and multiplication mod p — with no floating-point unit, no rounding, no denormalized numbers. This bridging proceeds in three steps.

First, scaling: choose $s = 2^k$ (typically k = 7 to 16), compute $w_q = \text{round}(w \times s)$. The scale must be large enough that rounding error is small but small enough that accumulated sums across thousands of neurons don't approach the field modulus. Second, rescaling after multiplication: when two scaled values multiply, the result has scale s^2. Returning to scale s requires proving a division-with-remainder: $q \times s + r = y_{scaled}$ with $0 \le r < s$ — ZEN's "remainder verification"

gadget, costing k constraints per range check. Third, zero-point and asymmetric quantization: some schemes use $w_q = \text{round}(w \times s) + zp$, common in TFLite (uint8) and ONNX Runtime's static quantization, with dequantization $w_{float} = (w_q - zp) \times \text{scale}$. The zero-point adds a subtraction constraint per weight and must be committed alongside the weights.

This three-step process is the central interface between ML and ZK. EZKL exposes it as a configurable scale parameter with a calibrate step for per-layer optimization. ZKML [7] starts from pre-quantized int8 TFLite models. zkLLM uses Q=16 with lookup-based bounds checking. Our PR #1025 (Section 8) encountered this interface directly: ONNX Runtime's PTQ inserts explicit QuantizeLinear/DequantizeLinear ops that EZKL must strip because it handles quantization internally.

Why neural networks are hostile to ZK circuits

Matrix multiplication dominates inference FLOPs (>90%) and is the most ZK-friendly neural-network operation — pure field-native arithmetic. A transformer layer's QKV projection (~1.77M multiplication constraints at $d = 768$) is large but tractable, especially since Freivalds' algorithm (Section 7.2) can verify matmuls probabilistically in $O(n^2)$ rather than proving them in $O(n^3)$.

Nonlinear activations are where cost explodes. ReLU ($\max(0, x)$) requires sign-bit decomposition in $\mathbb{F}_p$ — prove (p, n) with $x = p - n$, $p \times n = 0$, and range checks on both — costing ~64 constraints per activation. VGG-16's 15 million ReLUs add ~960 million constraints for nonlinearity alone. Softmax is worse: exp() has no polynomial representation over $\mathbb{F}_p$, forcing polynomial approximation (accuracy degrades with depth), piecewise-linear lookup tables, or zkLLM's digit decomposition. Layer normalization requires non-native division, in-circuit square root, and normalization — hundreds of constraints per element.

A transformer's attention — two $O(n^2 d)$ matmuls, $\sqrt{d}$ division, per-row softmax — is the single most ZK-hostile ML primitive, which is why zkLLM required a bespoke protocol (zkAttn + tlookup) just to make it tractable.

Classical ML: orders of magnitude cheaper

Decision trees are the ideal ZK-friendly model. Each node requires one comparison (~k constraints for k-bit values) and one multiplexer (~3 constraints), with no multiplication. A depth-10 path costs ~200 constraints — less than four ReLU activations. zkDT [41] proved a 1029-node tree in 250 seconds with a 287 KB proof. Random forests scale linearly and independently: a 500-tree forest at depth 10 costs ~100,000 constraints — comparable to a single dense layer — with accuracy matching DNNs on tabular data [42].

Logistic regression costs d multiplication constraints for the dot product plus one sigmoid (applied once, not millions of times). Garg et al. [12] prove training of a 1024-feature logistic regression on 262k records in ~10 minutes. Linear SVMs match logistic regression; kernel

SVMs are harder (per-support-vector exp() or polynomial evaluations). Naive Bayes is cheap (field multiplications). k-NN requires committed-sort at O(n log n).

EZKL's position in this landscape

EZKL's op support reflects the gradient directly. Linear operations (MatMul, Conv, Gemm, Add, Mul, BatchNorm) are polynomial gates. Nonlinearities (ReLU via decomposition, Sigmoid/Tanh/Softmax via Halo2 lookup arguments) are lookup tables, with logrows controlling the accuracy-versus-cost tradeoff. For classical ML models exported via skl2onnx, circuits consist almost entirely of cheap operations and prove in seconds. For transformers, lookup-table usage explodes, proving reaches minutes to hours, and circuit splitting becomes necessary — the circuit-friendliness gulf made concrete in a production system.

The lookup-argument response

The lookup-argument literature exists to close the non-linearity gap: Plookup [37], cq [38] (prover independent of table size after preprocessing), LogUp [39] (logarithmic-derivative-based), and Lasso [23] (sub-linear prover via virtual tables with SOS decomposition). An emerging architecture-side response replaces softmax entirely — PolySketchFormer [40] shows degree-4 polynomial attention matching softmax from scratch, eliminating zkML's worst primitive.

Practical implications

On tabular data, decision-tree ensembles are 100–1000× cheaper to prove than accuracy-matched neural networks, and since tree ensembles routinely match or beat DNNs on tabular tasks [42], they are the default for any ZK-budgeted pipeline. Spectral Finance's MACRO credit score ships on Ethereum using EZKL precisely because shallow MLPs and tree models fit its on-chain verification cost envelope where a transformer never would.

Cut-and-Choose, Freivalds, and the Probabilistic Middle Ground

Between full ZK proofs and pure trust sits a family of probabilistic verification schemes often better matched to ML workloads than SNARKs.

Cut-and-choose

The paradigm traces to Rabin [45] and was formalized by Lindell and Pinkas [46]. Applied to ML training, the prover commits to every training iterate, the verifier challenges a random subset, and SNARKs are produced only for opened samples — VeriML [51] and zkMLaaS [52] apply this template directly. With k = 40 samples over a 256-bit field, cheating probability drops below 2^(-40), converting the astronomical cost of a full proof of training into the cost of a small number of spot-checked proofs.

Freivalds' algorithm

Freivalds' algorithm [47] verifies AB = C by picking random r and checking A(Br) = Cr — $O(n^2)$ rather than O(n^ω), with error 1/|𝔽| per round. Over a 64-bit field, one round gives 2^(-64) soundness. Since neural-network inference is >95% matrix multiplication, Freivalds directly verifies the dominant operation orders of magnitude cheaper than SNARK compilation. Thaler's GKR-sumcheck matmul IP [48] generalizes this through multilinear extensions and is the spine of every sumcheck-based zkML system. The practical rule of thumb: sumcheck/Freivalds incurs ~5–50× prover overhead over native inference, while SNARK-compiled NN inference incurs 10^4–10^6× — explaining why GKR dominates the frontier.

Optimistic verification and the pluralistic resolution

opML [49] transplants Arbitrum's fraud-proof machinery onto ML via a MIPS-like VM with interactive bisection following TrueBit [50], demonstrating 7B-LLaMA on commodity CPUs — far beyond current zkML — at the cost of a 7-day security window and honest-challenger assumption. The field is resolving the optimistic-vs-pessimistic debate pluralistically: Ritual's Symphony consensus exposes four sidecars (ZKML, opML, PPML probabilistic spot-checking, and TEE attestation), letting developers match verification guarantees to application requirements. No single method dominates the cost-latency-trust space.

An Open-Source Contribution to EZKL: PR #1025

Why EZKL, and why this issue

A thorough audit of every major open-source zkML repository — ddkang/zkml, uiuc-kang-lab/zk-torch, Lagrange-Labs/deep-prove, gizatechxyz/LuminAIR, gizatechxyz/orion, and zkonduit/ezkl — revealed that EZKL is the only zkML project with verifiably continuous public open-source activity as of April 2026. Every other codebase has gone quiet on GitHub: academic publication cycles (ddkang/zkml, zk-torch), defense-contract pivots to private repos (DeepProve), team pivots to product/token work (LuminAIR), or explicit archival (Orion, Worldcoin's proto-neural-zkp). This finding — that the open-source engineering ecosystem has not kept pace with research output — drove the contribution strategy.

Issue #942 reported that a user who had applied ONNX Runtime's post-training quantization to a HuggingFace face-landmark model (unity/sentis-face-landmarks) received a cryptic crash: [E] [tract] Failed analyse for node #197 "conv2d_1_quant" ConvHir. The root cause was an impedance mismatch between two quantization paradigms — ONNX Runtime's PTQ inserts explicit QuantizeLinear, DequantizeLinear, DynamicQuantizeLinear, MatMulInteger, and ConvInteger ops into the graph, but EZKL already performs its own quantization internally via the scale parameter (Section 6.1), so the user's pre-quantization was both redundant and unsupported. This is not merely an EZKL implementation gap — it is a concrete instance of the float-to-field boundary problem that defines zkML engineering.

Detection and reporting

The first component added a GraphError::UnsupportedQuantizationOps(String) variant and a scanner that runs immediately after tract's initial parse, before the typing/decluttering passes that crash. The scanner matches op names against the full PTQ family (QuantizeLinear, DequantizeLinear, DynamicQuantizeLinear, QLinear, ConvInteger, MatMulInteger), reports up to 5 offending nodes, and suggests exporting the original floating-point model. Two test fixtures were generated from scratch — a QuantizeLinear → DequantizeLinear → Conv graph and a DynamicQuantizeLinear → MatMulInteger → Cast → Mul graph — because all existing EZKL tests used float ONNX models.

The architectural pivot: from Python script to in-process Rust rewrite

The initial design was a Python preprocessing script that walks the ONNX graph, strips Q/DQ patterns, and writes a cleaned .onnx to disk. It worked, but an architectural question reshaped the contribution: "do we already walk the ONNX graph? could we do it on the spot when Q/DQ pairs are encountered?" This collapsed the detector, the Python script, and the hypothetical tract-level rewrite into one in-process Rust pass running transparently inside Model::new. The user feeds their pre-quantized model to ezkl gen-settings and it works — no manual step, no Python dependency. The detector survives as a safety net for patterns the rewriter doesn't yet handle. The original Python script design required a manual preprocessing step, two implementations to maintain, and left the in-process rewrite as future work; the redesigned approach auto-rewrites transparently in pure Rust, callable from both the auto-pass and a standalone ezkl dequantize subcommand.

The three rewrite patterns

The src/graph/dequantize.rs module (~430 lines, 8 unit tests) canonicalizes three patterns at the protobuf level via prost:

Activation Q/DQ identity pairs: when a QuantizeLinear feeds directly into a DequantizeLinear with the same scale and zero-point, the pair is the identity modulo quantization noise. Both nodes are removed and the consumer is rewired to the Q node's input — recognizing that round(x × s) / s ≈ x, an identity that EZKL's own quantization will supersede.

Standalone weight DequantizeLinear: when an int8 weight initializer feeds through DequantizeLinear into a downstream op, the pass computes W_float = (W_int − zero_point) × scale, stores it as a new float TensorProto, and rewires the op. This is the float-to-field bridge from Section 6.1 performed once at load time.

DynamicQuantizeLinear + integer-op fusion: the pattern ONNX Runtime's quantize_dynamic actually emits. A five-node subgraph (DynamicQuantizeLinear → scale/zp → IntegerOp → Cast → Mul) collapses to a single float op with dequantized weights. Spatial attributes on ConvInteger are preserved on the replacement Conv. A bug found during testing — when one

DynamicQuantizeLinear fans out to multiple integer ops, the producer-lookup filtered already-dropped nodes — was fixed by walking forward from the integer op using a node index built once at the start.

Testing and results

Default tests run mock-only and complete in seconds; the heavy setup → prove → verify variant behind #[ignore] validated a 19 KB KZG proof in 3.87 seconds. Adding the quantized fixture to the existing test array got it picked up by 36 existing wrappers with a witness max-abs error of 2.4×10^{-4}. End-to-end on the issue's 86-node face-landmark model, ezkl gen-settings succeeds transparently (45 fusions auto-rewritten, 0.02 s overhead); with --disable-quantization-fixup, the safety net fires with an actionable error listing 44 PTQ nodes. PR #1025 was submitted with ~430 lines of new code, 3 test files, 2 ONNX fixtures, and modifications to loader, CLI, config, and Python bindings — all green on cargo clippy, cargo fmt, and the full suite.

What this contribution teaches about zkML

Three observations connect this contribution to the report's broader themes. First, the float-to-field boundary is where zkML actually lives. This issue was not cryptographic but representational — EZKL's scale parameter, ZEN's remainder verification, zkLLM's Q=16 lookups, and ONNX Runtime's PTQ are all different answers to the same question: how do you bridge floating-point ML to integer arithmetic over a prime field? The key insight generalizes: pre-quantization is the wrong abstraction for ZK, because PyTorch/ORT quantize for inference-engine performance (int8 bandwidth) while ZK provers care about field arithmetic and lookup-table sizes — a completely different cost model.

Second, the maintainability-versus-capability tradeoff recurs throughout zkML toolchain design. A circuit-level approach would unlock proving quantized models as quantized, preserving tighter lookup ranges, but would touch the constraint compiler and op-dispatch system. The protobuf-level approach keeps the circuit compiler untouched — the right call for a stable codebase. That same tradeoff separates zkLLM's custom protocols from EZKL's compiler approach.

Third, the PTQ impedance mismatch is a neural-network-specific problem. A decision-tree model exported via skl2onnx would never trigger this bug — decision trees use no QuantizeLinear ops, no DynamicQuantizeLinear fusions, and no MatMulInteger nodes. Their ONNX graphs consist entirely of comparison and selection ops that tract parses without difficulty. Weight quantization for inference acceleration has no analog in the random forest or logistic regression world, making this the circuit-friendliness gulf (Section 6.4) made concrete in a bug report.

The broader open-source landscape

Every other major zkML codebase has gone quiet on public GitHub: ddkang/zkml (superseded by ZKTorch), Lagrange-Labs/deep-prove (stalled mid-2025 as Lagrange shifted to private repos

for defense contracts), gizatechxyz/LuminAIR (stagnant since September 2025, team pivoted to product/token work), gizatechxyz/orion (archived, replaced by LuminAIR), and zkLLM (paper artifact only). Orion's archival validates that ONNX-to-circuit compilation (EZKL) and computation-graph-native proving (DeepProve's GKR) beat running inference inside a general-purpose zkVM for any model beyond toy scale. Research advances monthly, but the engineering ecosystem is concentrated in essentially one continuously maintained project — a gap between research velocity and open-source infrastructure maturity that is itself a significant finding.

Conclusion

zkML has crossed from impossibility to practicality for small-to-medium models. The arc from SafetyNets [1] to zkLLM's 13B forward pass [8] and ZKTorch's universal compiler [9] took seven years. Production deployments — Worldcoin's iris proofs, Spectral's on-chain credit scores via EZKL, ORA's optimistic oracles — demonstrate that the engineering works. Consumer GPUs prove Ethereum blocks in real time (Section 5), and the circuit-friendliness gulf (Section 6) is well enough understood that practitioners can choose the right model family for their verification budget.

Four things have not changed. No GPT-3-scale proof exists. Every zkML system on Ethereum is quantum-vulnerable at its on-chain Groth16/KZG wrapper (Section 4). The STARK family — post-quantum-safe, powering the fastest provers — has no working zkML framework at transformer scale (Orion failed, LuminAIR stalled), making this the single most consequential open problem in zkML infrastructure. And the open-source ecosystem is concentrated in one project: EZKL (Section 8.7).

The contribution work (Section 8) surfaced two findings beyond the literature survey: the float-to-field quantization boundary is where zkML's central engineering challenge lives, and the PTQ impedance mismatch is neural-network-specific with no classical-ML analog. The verification modality question resolves as application-specific rather than open — full ZK, cut-and-choose, optimistic, or TEE, dialed per use case (Section 7).

The synthesis that would matter most — ZK-verifiable watermarked inference under post-quantum-safe STARK proofs on purpose-built silicon, routed through a decentralized prover marketplace — has every ingredient in the 2024–2026 literature but none composed end-to-end. That composition is the research that will matter in five years.

Github Pull Request

https://github.com/zkonduit/ezkl/pull/1025

AI agent chat

Usage revolves around one single chat and an associated coding session. The chat is at...oops, Claude Desktop tells me: “Chats using advanced research can't be shared”. I will

readily export it or copy and paste upon request. The coding session report is at https://drive.google.com/file/d/1gagzJmdUS1C98lKo6B-joL98q-ZyC7vj

References

[1] Ghodsi, Z., Gu, T., Garg, S. "SafetyNets: Verifiable Execution of DNNs on an Untrusted Cloud." NeurIPS 2017.

[2] Goldwasser, S., Kalai, Y.T., Rothblum, G.N. "Delegating Computation: Interactive Proofs for Muggles." STOC 2008.

[3] Feng, B., Qin, L., Zhang, Z., Ding, Y., Chu, S. "ZEN: An Optimizing Compiler for Verifiable, Zero-Knowledge Neural Network Inferences." ePrint 2021/087.

[4] Liu, T., Xie, X., Zhang, Y. "zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy." CCS 2021.

[5] Weng, C., Yang, K., Xie, X., Katz, J., Wang, X. "Mystique: Efficient Conversions for Zero-Knowledge Proofs with Applications to Machine Learning." USENIX Security 2021.

[6] Kang, D., Hashimoto, T., Stoica, I., Sun, Y. "Scaling up Trustless DNN Inference with Zero-Knowledge Proofs." arXiv:2210.08674, 2022.

[7] Chen, B., Waiwitlikhit, S., Stoica, I., Kang, D. "ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs." EuroSys 2024.

[8] Sun, H., Li, J., Zhang, Y. "zkLLM: Zero Knowledge Proofs for Large Language Models." CCS 2024; arXiv:2404.16109.

[9] Chen, B., Tang, H., Kang, D. "ZKTorch: Compiling ML Inference to Zero-Knowledge Proofs via Parallel Proof Accumulation." arXiv:2507.07031, 2025.

[10] Huang, Y. et al. "zkPyTorch: A Hierarchical Optimized Compiler for Zero-Knowledge Machine Learning." ePrint 2025/535.

[11] Qu, Z. et al. "zkGPT: An Efficient Non-interactive Zero-knowledge Proof Framework for LLM Inference." USENIX Security 2025; ePrint 2025/1184.

[12] Garg, S., Goel, A., Jha, S. et al. "Experimenting with Zero-Knowledge Proofs of Training." CCS 2023; ePrint 2023/1345.

[13] Abbaszadeh, A., Pappas, C., Katz, J., Papadopoulos, D. "Kaizen: Practical Zero-Knowledge Proofs of Training." CCS 2024; ePrint 2024/162.

[14] Sun, H., Zhang, Y. "zkDL: Efficient Zero-Knowledge Proofs of Deep Learning Training." 2023.

[15] Watanabe, R., Uchikoshi, K. "Proof of Prompt." WWW 2025.

[16] Christ, M., Gunn, S., Zamir, O. "Undetectable Watermarks for Language Models." COLT 2024; ePrint 2023/763.

[17] Kothapalli, A., Setty, S., Tzialla, I. "Nova: Recursive Zero-Knowledge Arguments from Folding Schemes." CRYPTO 2022.

[18] Kothapalli, A., Setty, S. "HyperNova: Recursive Arguments for Customizable Constraint Systems." CRYPTO 2024.

[19] Bünz, B., Chen, B. "ProtoStar: Generic Efficient Accumulation/Folding for Special-Sound Protocols." ASIACRYPT 2023.

[20] Wu, H., Zheng, W., Chiesa, A., Boneh, D., Saxena, N. "DIZK: A Distributed Zero Knowledge Proof System." USENIX 2018.

[21] Liu, T., Xie, X., Zhang, Y., Song, D., Zhang, Y. "Pianist: Scalable zkRollups via Fully Distributed Zero-Knowledge Proofs." IEEE S&P 2024.

[22] Xie, T., Zhang, Y., Song, D. "Libra: Succinct Zero-Knowledge Proofs with Optimal Prover Computation." CRYPTO 2019.

[23] Setty, S., Thaler, J., Wahby, R. "Lasso and Jolt: SNARKs via Lookups." EUROCRYPT 2024.

[24] Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M. "Fast Reed-Solomon IOP of Proximity (FRI)." ICALP 2018.

[25] Ben-Sasson, E. et al. "DEEP-FRI." ITCS 2020.

[26] Arnon, G., Chiesa, A., Fenzi, G., Yogev, E. "STIR: Reed-Solomon Proximity Testing with Fewer Queries." CRYPTO 2024.

[27] Arnon, G., Chiesa, A., Fenzi, G., Yogev, E. "WHIR: Reed-Solomon Proximity Testing with Super-Fast Verification." ePrint 2024/1586.

[28] Diamond, B., Posen, J. "Binius: Efficient Proofs over Binary Fields." ePrint 2023/1784.

[29] Ames, S. et al. "Ligero: Lightweight Sublinear Arguments Without a Trusted Setup." CCS 2017.

[30] Ben-Sasson, E. et al. "Aurora: Transparent Succinct Arguments for R1CS." EUROCRYPT 2019.

[31] Golovnev, A., Lee, J., Setty, S., Thaler, J., Wahby, R. "Brakedown: Linear-Time and Field-Agnostic SNARKs for R1CS." CRYPTO 2023.

[32] Xie, T., Zhang, Y., Song, D. "Orion: Zero Knowledge Proof with Linear Prover Time." CRYPTO 2022.

[33] Beullens, W., Seiler, G. "LaBRADOR: Compact Proofs for R1CS from Module-SIS." CRYPTO 2023.

[34] Nguyen, N.K., Seiler, G. "Greyhound: Fast Polynomial Commitments from Lattices." CRYPTO 2024.

[35] Boneh, D., Chen, B. "LatticeFold: A Lattice-Based Folding Scheme." 2024.

[36] Groth, J. "On the Size of Pairing-Based Non-interactive Arguments." EUROCRYPT 2016.

[37] Gabizon, A., Williamson, Z.J. "Plookup: A Simplified Polynomial Protocol for Lookup Tables." ePrint 2020/315.

[38] Eagen, L., Fiore, D., Gabizon, A. "cq: Cached Quotients for Fast Lookups." ePrint 2022/1763.

[39] Haböck, U. "LogUp: Multivariate Lookups in Sumcheck-Based IOP." ePrint 2022/1530.

[40] Kacham, P., Mirrokni, V., Zhong, P. "PolySketchFormer: Fast Transformers via Sketching Polynomial Kernels." ICML 2024.

[41] Zhang, Y., Fang, Z., Zhang, Y., Papamanthou, C. "zkDT: Zero Knowledge Decision Tree Predictions and Accuracy." CCS 2020.

[42] Grinsztajn, L., Oyallon, E., Varoquaux, G. "Why Do Tree-Based Models Still Outperform DNNs on Tabular Data?" NeurIPS 2022.

[43] Lu, W. et al. "cuZK: Accelerating Zero-Knowledge Proof with A Faster Parallel Multi-Scalar Multiplication Algorithm on GPUs." TCHES 2023.

[44] Ma, H. et al. "GZKP: A GPU Accelerated Zero-Knowledge Proof System." ASPLOS 2023.

[45] Rabin, M.O. "How to Exchange Secrets with Oblivious Transfer." Harvard TR-81, 1981; ePrint 2005/187.

[46] Lindell, Y., Pinkas, B. "An Efficient Protocol for Secure Two-Party Computation in the Presence of Malicious Adversaries." EUROCRYPT 2007.

[47] Freivalds, R. "Probabilistic Machines Can Use Less Running Time." IFIP Congress 1977.

[48] Thaler, J. *Proofs, Arguments, and Zero-Knowledge.* Textbook, 2022.

[49] Conway, S. et al. "opML: Optimistic Machine Learning on Blockchain." arXiv:2401.17555, 2024.

[50] Teutsch, J., Reitwießner, C. "TrueBit: A Scalable Verification Solution for Blockchains." arXiv:1908.04756.

[51] Zhao, L. et al. "VeriML: Enabling Integrity Assurances and Fair Payments for Machine Learning as a Service." IEEE TPDS 2021; arXiv:1909.06961.

[52] Huang, T. et al. "zkMLaaS: A Verifiable Scheme for Machine Learning as a Service." IEEE GLOBECOM 2022.